

control over the server. The correct way to configure this is by creating a specific user account with only required permissions and using that user to run the MySQL service. This further helps in auditing and logging the MySQL service.

Secure service binding: In the MySQL configuration file, you can use the following command to bind the service to a specific IP address. This is essential, because any additional IP address configured on the machine can start publishing the MySQL service by default, which can lead to exposed vulnerabilities.

```
bind-address = 127.0.0.1 #(or other machine_ip)
```

Secure root user: It is a common practice for administrators to configure the database server, but forget to disable default accounts—or at least to change the passwords. The root or admin user password is a critical setting, because leaving it vulnerable can lead to serious attacks. Another common mistake is to leave the root password blank. Run this command on the MySQL server:

```
mysql -u root
```

If you are able to connect, it means that the password is blank. A stronger password policy should be implemented for the root, as well as other database users. This helps achieve security against password guessing and brute-force attacks.

Remove default users: A MySQL installation typically creates a few anonymous users by default. On successful installation, these users have no reason to remain in the system, because their presence can potentially result in a back-door for attackers.

Remove default files: On all open source distributions, MySQL creates a sample database found under *user/db*. While there is no sensitive information in it to steal, this database accepts anonymous connections, and hence can act as a backdoor for attackers. Ideally, drop this database. Then, there's the history file (*~/.mysql_history*), which will contain the installation and configuration steps you performed, which can expose the passwords of critical database users. If you want a copy, make a secure off-machine back-up and then delete this file.

Change the default port: By default, MySQL listens on TCP port 3306. For database servers hosted and being used internally, this may not be a problem, but for externally exposed ones, changing the default port is a good idea. Attackers typically use finger-printing tools, and figure out the services based on ports. Changing the port from default to an unknown or less-used one can make the attackers' task much more difficult.

Control host access: Typically, a MySQL server is installed to run as a standalone, while application servers and Web servers connect to it for data access. By using the *hosts.deny* and *hosts.allow* files, the access to the database server can be limited only to the Web and application servers. Omitting this step can potentially lead to LAN-based human or virus/spyware attacks, which usually target the database server to steal information.

SQL Injection: This attack is not a pure database attack, but is in fact a combination of a database and Web vulnerability. It's

a very well known type of attack, in which an attacker uses Web pages to plant the attack. It is done by including parts of SQL query statements in the Web form, which is usually to be filled up by Web users with relevant information. If the Web page is not capable of handling this situation in a secure manner, it ends up sending this bogus request to the database. The attackers can simply run a select query to dump the entire database on their machine by this method, which makes it a favourite of hackers and hence dangerous from a firm's security standpoint. To ensure protection from this type, all SQL queries must perform input data validation. Using triggers and constraints can also help achieve some level of security.

Security patching: Patching is a first level of defence for databases. A robust patching system must be deployed to secure all the servers connecting to a database server. There are two types of patches—one being OS-level security patches, and the other specific to database service security. System patching routines must ensure both types are addressed.

Protection from network-level attacks: The MySQL database can be prone to typical network-based attacks such as packet sniffing, packet crafting, man-in-the-middle, denial of service, etc. Implementing a robust firewall and Intrusion Detection Service is imperative. In addition to this, a logging and monitoring system must be incorporated too. It helps in finding out historic data leakage and data theft attacks, which are required for compliance purposes. Utmost care needs to be taken for MySQL installations hosted in a DMZ (demilitarised zone).

General guidelines for MySQL security

Since database servers are an important component of the whole system, multiple measures need to be deployed to ensure security. As for MySQL, it is advisable to stick to one authentication method—preferably the one provided by the MySQL service. When you don't use OS-based authentication, you automatically keep unauthorised users away, making the access-control job easier. As for roles, a minimum set of roles should be created to avoid future confusion, and the roles should be application-specific rather than purpose-specific. This helps in keeping track of which user did what, in terms of accessing the resources. Disabling unwanted MySQL database features and services is a good idea too.

In terms of data integrity, using SSL or TLS encryption is useful, especially to avoid packet sniffing of the data flowing between the requesting server and the database server. As for data encryption, there are built-in features available for the database server, whereby the encryption can be achieved for the entire database, or at a granular level such as for tables or schemas. This helps achieve data security without compromising on server performance. 

By: Prashant Phatak

The author has over 22 years of experience in the field of IT hardware, networking, Web technologies and IT security. Prashant runs his own firm named Valency Networks in India (<http://www.valencynetworks.com/>) providing consultancy in IT security design, security penetration testing, IT auditing, infrastructure technology and business process management. He can be reached at prashant@valencynetworks.com.